

DOMAIN 2 • JSON SCHEMA & TOOL DESIGN

If you declare multiple parameters in your tool's properties JSON, how do you make sure the model supplies all of them?

List all property keys inside the 'required' array of the JSON Schema. ✓

Include 'MUST INCLUDE ALL' in the system prompt.

Tool parameters are always optional.

Enable parallel tool options.

Explanation & Rationale

In JSON Schema, the properties defined are only mandatory if their keys are added to the 'required' list of the root parameters schema.

You want to dynamically test how temperature impacts the output variety. How should you design your prompt testing system in the Console Workbench?

Run multiple Workbench instances with different temperature settings under the same prompt scenario and compare outputs.



Variables are not supported for temperature values.

Temperature must be configured inside the prompt text.



Workbench only supports temperature 0.0.

Explanation & Rationale

The Console Workbench allows you to manually adjust parameters like temperature for each test run to compare results.

When defining tool descriptions in your JSON Schema, what is the best practice to help Claude select the correct tool?

Provide highly descriptive, clear, and unambiguous functional statements detailing what the tool does and when to call it.



Keep descriptions as short as possible to save tokens.

Write descriptions in uppercase.

Describe the tool code logic inside the system prompt instead.

Explanation & Rationale

Claude relies on tool descriptions and parameter descriptions to understand the tool's purpose and choose the correct parameters.

Why does Anthropic recommend using matching closing tags (e.g., `</docs>`) inside system prompts when referencing structured data?

It signals to the model exactly where the input context ends, preventing instructions from bleeding into subsequent segments. ✓

It reduces prompt execution latency.

It activates prompt caching protocols.

It is required by the HTTP compiler to parse the JSON body.

Explanation & Rationale

Closing tags provide explicit structures, allowing the model's self-attention layers to distinguish instructions from variables clearly.

How should prompt templates handle multiline dynamic data, such as code segments or files, inside prompts?

Wrap the dynamic string in dedicated XML tags (e.g. `<code_block>{{CODE_BLOCK}}</code_block>`) to prevent formatting leaks.



Replace all newline characters with empty spaces.

Encode the strings into base64 format.

Inject multiline files directly without delimiters.

Explanation & Rationale

XML tags handle multiline text gracefully, isolating content from instructions and preventing formatting leaks.

How does Claude handle prompt instruction overrides (jailbreaks) that are wrapped inside XML tags compared to plain text strings?

XML tags create clear semantic boundaries, making it easier for Claude to recognize dynamic data inputs and ignore injected overrides when instructed by the system prompt.



XML tags automatically trigger encryption protocols.

Claude rejects any message containing brackets (< or >).

Claude treats XML tags as system messages with highest priority.

Explanation & Rationale

Claude is pre-trained to parse XML tags as structured document boundaries, separating instructions from user-supplied data variables.

After Claude outputs a tool request, how must your application submit the tool execution results back to the Messages API?

Add a message to the messages array with the role 'user', containing a content block of type 'tool_result' that matches the 'tool_use_id'.



Call a separate HTTP POST endpoint `~/v1/tool_results``.

Inject the result into the system prompt parameters.

Modify the previous assistant message payload directly.

Explanation & Rationale

Tool results are submitted by appending a 'user' message to the thread containing a content block with type 'tool_result' and the matching 'tool_use_id'.

When parsing tool execution arguments generated by Claude, your code encounters a JSON syntax error. How should your application handle this gracefully?

Crash and report to the user.

Intercept the error and send a corrective message back to the assistant (e.g., 'Error: Invalid JSON format. Please output valid arguments.') to allow the model to self-correct.



Retry the request using temperature 1.0.

Use standard regex to strip all braces.



Explanation & Rationale

Providing structural error logs back to the model in the next turn is a standard self-healing agent pattern, allowing it to regenerate correct syntax.

Which parameter controls the maximum length of the dynamic system prompt?

There is no parameter limiting the system prompt; it is only bound by the overall 200k token context window of the model.



max_system_tokens

limit_system

system_tokens_limit



Explanation & Rationale

System prompts are part of the input context, limited only by the overall input context window of 200,000 tokens.

When using the Messages API, what parameter specifies the output token budget limit for Claude's response generation?

`max_tokens`



`limit_tokens`

`output_token_limit`

`response_budget`

Explanation & Rationale

The 'max_tokens' parameter dictates the maximum number of output tokens Claude is allowed to generate before halting.

When a tool execution fails inside your application, how should you communicate the error message back to Claude?

Send a message with role 'user', containing a 'tool_result' block with 'is_error: true' and the error details inside the 'content'.



Raise an HTTP 500 error code to Anthropic servers.

Send the error message inside the system prompt.

Shut down the chat thread.

Explanation & Rationale

Anthropic's tool result schema supports an 'is_error' boolean flag. Setting this flag to true tells Claude the execution failed, allowing it to adapt its logic.

Your application throws an error indicating 'anthropic.APIConnectionError'. What does this represent, and how should it be handled?

The client API key has expired.

A network connectivity issue occurred while reaching Anthropic servers. Catch the error and retry using backoff. ✓

The requested model name was typed incorrectly.

The request payload exceeded token limits.

Explanation & Rationale

APIConnectionError indicates network transit failures. Implementing retries with exponential backoff helps resolve transient drops.

You want Claude to return a tool call arguments array, but want to strictly block it from outputting parallel tool calls in the same turn. What setting resolves this?

Parallel tool calls cannot be disabled programmatically; developers must check and reject extra calls inside the client validation logic.



Set ``parallel_tools: false`` in headers.

Set ``tool_choice: { type: "single" }``.

Lower the temperature to 0.0.

Explanation & Rationale

The API does not have a flag to disable parallel tool calls. If parallel outputs are not supported by your backend, you must handle and validate them client-side.

You are designing a chatbot. You want to instruct Claude to adopt a specific communication style. Where should these guidelines be defined?

Inside the system parameter.



Inside the model temperature parameter.

At the beginning of each user message.

In the HTTP request headers.

Explanation & Rationale

System prompts are designed to hold formatting guidelines, rules, stylistic limits, and conversational guidelines.

Which Messages API parameter allows you to establish structural context, instructions, and rules before the conversation history begins?

messages[0].content

system



prompt_prefix

model_rules

Explanation & Rationale

The 'system' parameter is used to specify system prompts. This isolates structural rules from the user messages array.

If you want to prevent Claude from returning any text explanation alongside a tool call request, what parameter settings can assist?

Set `tool_choice` to type `tool` specifying the target tool name.



Set model temperature to 1.0.

Add a system prompt saying 'Only run tools, do not speak.'

There is no mechanism to prevent conversational preambles.



Explanation & Rationale

Forcing tool usage via `tool_choice: { type: "tool", name: "..."}` stops Claude from generating conversational preamble text.

How can you instruct Claude to output a valid XML document as its completion response?

Set temperature to 0.0.

Prefill the assistant response block with '<root>' to start the completion format. ✓

Wrap the system prompt in brackets.

Use base64 encoding.

Explanation & Rationale

Prefilling the assistant response starting token with `<root>` forces the model to complete the tag structure, forcing XML formatting.

You are defining a tool schema for a tool named `fetch_logs`. It requires a parameter `log_levels` which should be an array of strings (e.g., `['info', 'error']`). How do you specify this in JSON Schema?

```
{ "type": "array", "items": { "type": "string" } }
```



```
{ "type": "string[]" }
```



```
{ "type": "list", "contents": "string" }
```

```
{ "type": "array[string]" }
```

Explanation & Rationale

JSON Schema represents lists as `'type': 'array'`, and uses the `'items'` key to define the constraints on elements within that array.

Which parameter controls the deterministic nature of the model's outputs, and what setting should be used to guarantee identical results during API regression testing?

Set temperature to 0.0 to focus on maximum likelihood tokens.



Set temperature to 1.0 to enforce strict formatting rules.

Set max_tokens to 0.

Set stop_sequences to empty array.

Explanation & Rationale

A temperature of 0.0 makes the model's output generation deterministic, minimizing token selection variance across runs.

What client-side headers must be supplied to authenticate calls to the Anthropic Messages API?

Authorization: Bearer <key>

x-api-key: <key> and anthropic-version: 2023-06-01



api-token: <key>

x-anthropic-auth: Bearer <key>

Explanation & Rationale

Anthropic API requests require the 'x-api-key' header for credentials and the 'anthropic-version' header specifying the API schema version.

You are defining a JSON Schema for a tool called `fetch_records`. It has a parameter `record_ids` that must contain exactly an array of integers. What is the correct schema structure?

```
{ "type": "array", "items": { "type": "integer" } }
```



```
{ "type": "integer[]" }
```

```
{ "type": "array[int]" }
```

```
{ "type": "list", "items": { "type": "int" } }
```

Explanation & Rationale

JSON Schema represents arrays using 'type': 'array', and items inside the array are specified via the 'items' object containing the element type.

What happens if the model reaches the token count defined by the ``max_tokens`` parameter before completing its response?

The API triggers an HTTP 500 error code.

The response is truncated and the `stop_reason` is set to `'max_tokens'``. ✓

The API automatically bills for double the usage and completes the run.

The prompt is automatically re-run with lower temperature.

Explanation & Rationale

If ``max_tokens`` is hit, the model stops immediately. The resulting API payload has a ``stop_reason`` of ``max_tokens`` indicating truncation.

You want to design a tool schema that accepts an optional enum parameter. How should you specify this parameter in the properties block of your tool definition?

`{ "type": "string", "enum": ["high", "medium", "low"] }` without adding it to the required list. ✓

`{ "type": "string", "options": ["high", "medium", "low"], "required": false }` ✗

`{ "type": "string", "default": "medium" }` inside the system prompt.

Tool schemas do not support optional enum keys.

Explanation & Rationale

Standard JSON Schema represents optional parameters by leaving their keys out of the tool's 'required' list, and restricting options using the 'enum' property.

When sending dynamic user messages using the Messages API, which data type is required for the ``content`` parameter?

A raw string or an array of content blocks (such as text blocks or image blocks).



Strictly a base64 encoded string.

Strictly a JSON string.

An array of strings.

Explanation & Rationale

The Messages API supports both raw text strings and structured arrays of content blocks (for multimodality or structured tool responses).

Your application throws an HTTP 400 error indicating 'max_tokens' parameter is missing. What does this indicate?

You did not specify the max_tokens value in the request body.



The input prompt exceeded the max context limit.

The value of max_tokens was too high.

The API key was rate limited.

Explanation & Rationale

The 'max_tokens' parameter is a required field in every Messages API POST request.

When implementing streaming via the Messages API, which stream event containing metadata signals the creation of the message object?

message_start



message_delta

content_block_start

stream_opened

Explanation & Rationale

The 'message_start' event is the first event in the stream and contains the root message object, including the model name and role details.

What is the recommended design pattern when using variables inside prompt templates in your software application?

Isolate variables using custom XML tags and refer to those tags specifically in your instructions.



Always write variable values in base64 format.

Combine all variables into a single long string without tags.

Place variables inside the tools array.

Explanation & Rationale

Encapsulating variables in XML tags (e.g. `<data>{{DATA}}</data>`) ensures the model can identify boundaries and comply with instructions.

When Claude returns a tool execution request, does it perform the execution logic automatically?

No, Claude only output arguments and requests tool invocation. The developer's client application must execute the tool and send back the results.



Yes, Claude runs Python scripts internally to resolve tool requests.

Yes, Anthropic hosts serverless execution environments for tools.

No, tool execution is resolved by the browser runtime.

Explanation & Rationale

Claude does not execute tools. It determines *when* a tool should be run and *what* parameters to use, and delegates execution to the client application.

When utilizing prompt caching in the Messages API, which metadata blocks are eligible to receive cache flags?

The system parameter, tools configuration block, and dynamic items inside the messages array.



Only the model name parameter.

Only user messages containing file attachments.

The authorization header segment.

Explanation & Rationale

You can cache system prompts, tool declarations, or historical turns within the messages array by attaching cache control flags.

What is the default behavior of the Messages API if no system parameter is supplied in the request?

The model generates a response without any system prompt styling, behaving as a standard chatbot.



The request fails with an HTTP 400 error code.

The model defaults to a pre-defined system persona.



The api-version is set to legacy.

Explanation & Rationale

The system parameter is optional. If left out, Claude processes the user messages array using its default chat weights.

What does an HTTP status code of 403 represent when communicating with the Anthropic Messages API?

The request was throttled due to rate limits.

Authentication failed or client credentials/API keys are invalid.



The server is temporarily overloaded.

The model name specified does not exist.

Explanation & Rationale

403 Forbidden indicates authorization failures, such as invalid or expired API keys.

What HTTP status code is returned if your Messages API payload contains an invalid model parameter name?

400 Bad Request (invalid_request_error)



404 Not Found

401 Unauthorized

500 Internal Server Error

Explanation & Rationale

Typing errors in parameters (such as invalid model names) return an HTTP 400 Bad Request representing validation failures.

What is the token pricing structure of input vs. output tokens for Claude models?

Input tokens are cheaper than output tokens, making it efficient to supply rich context frames.



Input tokens cost the exact same as output tokens.

Output tokens are cheaper than input tokens.

Billing is based strictly on latency, not token count.

Explanation & Rationale

Like most LLM APIs, Anthropic bills output tokens (generation) at a higher rate per token than input tokens (prompt context).

When sending multiple messages in a conversation thread using the Messages API, what role sequence must your messages array adhere to?

Alternating user and assistant roles, starting with a user message.



Any order is acceptable, including consecutive assistant messages.

All user messages must be grouped at the beginning, followed by assistant replies.

All assistant messages must contain a 'system' key.

Explanation & Rationale

The Messages API requires messages to alternate between 'user' and 'assistant' roles, and the array must start with a 'user' turn.

Your application receives an HTTP 429 error code from the Messages API. What does this signify, and how should it be managed?

Authentication failed; refresh the API key.



Rate limit exceeded; implement exponential backoff with jitter and retry.



The prompt was too long; truncate the token counts.

Internal server error; report the issue to Anthropic.

Explanation & Rationale

429 represents rate limits. Retry strategies using exponential backoff with randomized jitter distribute requests smoothly over time.

You need to inject dynamic user-supplied data into a prompt template, but want to prevent Claude from interpreting the data as instruction overrides. What is the cleanest isolation format?

Put the user data inside double quotes.

Enclose the variable in XML tags (e.g., `<user_input>{{USER_INPUT}}</user_input>`) and refer to those tags in the system prompt. 

Convert the user data into lowercase.

Remove all space characters from the input string.

Explanation & Rationale

Isolating raw data inside XML tags and explicitly referencing those tags (e.g., 'Analyze only the contents within `<user_input>` tags') prevents prompt injections.

If you receive a 401 status code from the Messages API, what is the most likely cause?

The API key supplied is invalid, missing, or typed incorrectly.



Your rate limits were exceeded.

The server is temporarily overloaded.

The payload contains malformed JSON.

Explanation & Rationale

An HTTP 401 Unauthorized status indicates that the request lacked valid authentication credentials.

How should an application handle a scenario where Claude returns a tool call request for a tool that is not in the declared tool array?

Treat it as an execution bug, return a failure message block back to Claude, and log the incident.



Attempt to execute a random script matching the name.

Instantly shut down the server.

Change the model parameter to Opus.

Explanation & Rationale

If Claude hallucinations target non-existent tools, developers should return an error block in the next turn to allow self-correction.

To allow Claude to use multiple custom tools during a single response turn, what schema configuration adjustments are needed?

No changes are needed; Claude 3.5 Sonnet supports multiple tool call responses in a single turn by default if they are declared in the tools array.



Set `allow_parallel_tools` to true in the API header.

Each tool must be wrapped in a separate HTTP request payload.

Configure `tool_choice` to 'parallel_forced'.

Explanation & Rationale

Claude models natively support parallel tool use, allowing them to output multiple 'tool_use' blocks in a single content array turn.

What is the primary difference between a 429 rate limit error and a 529 server overloaded error in the Anthropic API?

429 means your client exceeded request or token limits; 529 means Anthropic's servers are experiencing high traffic.



429 means an invalid key; 529 means the payload was too large.

Both represent the same error condition.

429 is a client validation error; 529 is a billing failure.

Explanation & Rationale

429 indicates client quota limits were hit. 529 represents internal server overload, which is resolved by server scaling.

Which header version flag is currently required to access the Anthropic Messages API?

anthropic-version: 2023-06-01



anthropic-version: v3.5

api-version: 2024-01-01

x-version: standard-latest

Explanation & Rationale

The API version header value '2023-06-01' is required to call Messages API endpoints.

What schema format is officially supported by Anthropic to declare tool parameters in the Messages API payload?

XML Schema definition (XSD)

JSON Schema draft-07



GraphQL Type Schema

TypeScript Interface declarations

Explanation & Rationale

Anthropic utilizes standard JSON Schema formats to define the structure, properties, and constraints of parameters passed to tools.

Which model of the Claude 3.5 family should be used for high-intelligence coding assignments, multi-stage reasoning, and data analysis tasks?

Claude 3.5 Haiku

Claude 3.5 Sonnet



Claude 3.0 Opus



Claude 2.0

Explanation & Rationale

Claude 3.5 Sonnet represents the highest intelligence model in the 3.5 lineup, optimized for complex engineering, math, and reasoning.

What is the token cost pricing strategy when reusing a cached block of context?

Cached input tokens cost a fraction of the price of normal input tokens, drastically reducing costs for long sessions.



Cached tokens are billed at double cost.

Cached tokens are free.

Pricing remains the same as standard input tokens.

Explanation & Rationale

Anthropic offers prompt caching where read operations from the cache are priced significantly cheaper than standard input processing.

You want to inject dynamic reference files into the prompt. Which XML structure represents the cleanest layout for Claude?

```
<documents><document index="1"><source>{{SOURCE}}</source><content>{{CONTENT}}</content></document></documents>
```



```
--- FILE 1 --- {{CONTENT}} --- END FILE ---
```

```
Document content: {{CONTENT}}
```

```
[[document: {{CONTENT}}]]
```

Explanation & Rationale

Wrapping multiple documents in nested XML tags allows Claude to reference documents specifically by index or metadata (e.g. 'Refer to doc 1').

Your backend app gets a '504 Gateway Timeout' error when sending an extremely complex prompt to Claude. What is the first troubleshooting step?

Assume the API key has expired and request a new one.

Review client-side network timeout configurations and check if the prompt's complexity is stalling token generation.



Immediately double the size of the request batch.

Switch to using Opus for faster speeds.

Explanation & Rationale

A 504 error typically means the proxy or client closed the connection before the API responded. Adjusting timeouts or moving to a faster model (like Sonnet/ Haiku) helps.

Your server receives a '529 Overloaded' error from the Anthropic API. What does this indicate, and what action should your client-side integration perform?

The request size is too large; reduce the token count.

The Anthropic servers are experiencing high traffic; retry the request after a delay using exponential backoff.



The API key has been throttled permanently.

The model has timed out; switch to Opus.

Explanation & Rationale

529 signifies temporary server overload. Clients should handle this gracefully by retrying the request with exponential backoff and jitter.

You are building an application that handles large file uploads. What is the maximum input context window size supported by Claude 3.5 Sonnet?

100,000 tokens

200,000 tokens



80,000 tokens

500,000 tokens

Explanation & Rationale

Claude 3.5 Sonnet features a 200,000 token context window, allowing developers to process extensive files, code repositories, or histories.

You want to prefill the model response to guarantee JSON structure output. What sequence of keys represents the correct payload setup in the messages array?

Last message: { role: "assistant", content: "{ " }



System prompt: "Output raw JSON starting with {"

First message: { role: "system", content: "{ " }

Header parameter: { format: "json" }

Explanation & Rationale

Ending the messages array with an assistant message containing the opening character `{` forces the model to complete the JSON payload directly.

You want to force Claude to call a specific tool in your tool array, regardless of what the user says. Which Messages API field controls this?

`tool_choice: { type: "tool", name: "my_specific_tool" }`



`required_tool: "my_specific_tool"`

`force_call: true`

`tool_choice: { type: "any" }`

Explanation & Rationale

Setting `tool_choice` to type `tool` with the exact tool name forces Claude to output arguments for that tool instead of a text turn.

You want Claude to output answers in a custom XML format. How should you prefill the assistant response to ensure it adheres to this format?

Start the assistant message with '<response>' to force the model to output content matching your XML tags.



Specify '<xml>' in the model temperature field.

Add a stop sequence for '</response>'.

Inject brackets directly into the system prompt prefix array.

Explanation & Rationale

Prefilling response arrays starting with '<response>' forces Claude to continue the XML block structure without conversational headers.

Your API call fails with a 400 Bad Request error. The error message indicates 'invalid_request_error: system: System prompt must be a string or array of content blocks'. What payload formatting issue caused this?

The 'system' parameter was supplied inside the messages array instead of at the root level of the payload.



The system prompt exceeded the token limit.

The messages array contained a system role.

You did not specify a model parameter.

Explanation & Rationale

The 'system' parameter is a root-level property of the Messages API request body, not an entry in the 'messages' array.

If your Messages API response returns ``stop_reason: "stop_sequence"``, what does this indicate about the text generation process?

The model encountered an error during generation.

The model encountered one of the custom sequences defined in the ``stop_sequences`` array parameter and halted output.



The token window limit was exhausted.

The assistant turn completed normally (``end_turn``).

Explanation & Rationale

When one of your custom-defined stop strings (such as a newline or custom tag) is generated, Claude halts execution and reports 'stop_sequence'.

How should a developer monitor rate limits and remaining usage quotas programmatically?

Inspect rate limit headers (such as `anthropic-ratelimit-requests-remaining` and `anthropic-ratelimit-tokens-remaining`) returned in the HTTP response.



Call the `/v1/rate_limits` endpoint hourly.

Rate limits are only visible in the Console dashboard.

Estimate usage client-side by multiplying character counts.

Explanation & Rationale

The Anthropic API returns headers indicating current rate limits, remaining requests, and remaining tokens after each call.

You are setting up variables in the Claude Console Workbench. What syntax defines a variable placeholder in the prompt template?

`{{VARIABLE_NAME}}`



`$VARIABLE_NAME`

`[VARIABLE_NAME]`

`{VARIABLE_NAME}`



Explanation & Rationale

Double curly braces `{{...}}` are used in the Workbench console to declare prompt variables.

When configuring a tool schema in the Messages API, how do you define properties that the model **MUST** supply during tool calls?

Specify them in the prompt instruction text.

Add their property keys to the 'required' array in the parameters JSON schema. ✓

Set the property type to 'mandatory'.

Prefix the property names with an underscore.

Explanation & Rationale

JSON Schema standard dictates that required parameters must be listed inside the 'required' array of the properties object.

You are integrating Claude into an application that streams responses. What event marks the absolute end of the message streaming lifecycle?

content_block_stop

message_stop



message_delta

stream_finished

Explanation & Rationale

In the Messages API server-sent event (SSE) stream, 'message_stop' is the final event indicating that the message is complete and the stream is closing.

Which parameter controls the deterministic nature of the model's outputs, and what setting should be used to guarantee identical results during API regression testing?

Set temperature to 0.0 to focus on maximum likelihood tokens.



Set temperature to 1.0 to enforce strict formatting rules.

Set max_tokens to 0.

Set stop_sequences to empty array.

Explanation & Rationale

A temperature of 0.0 makes the model's output generation deterministic, minimizing token selection variance across runs.

You are integrating Claude into an application that streams responses. What event marks the absolute end of the message streaming lifecycle?

content_block_stop

message_stop



message_delta

stream_finished

Explanation & Rationale

In the Messages API server-sent event (SSE) stream, 'message_stop' is the final event indicating that the message is complete and the stream is closing.

When sending multiple messages in a conversation thread using the Messages API, what role sequence must your messages array adhere to?

Alternating user and assistant roles, starting with a user message.



Any order is acceptable, including consecutive assistant messages.

All user messages must be grouped at the beginning, followed by assistant replies.

All assistant messages must contain a 'system' key.

Explanation & Rationale

The Messages API requires messages to alternate between 'user' and 'assistant' roles, and the array must start with a 'user' turn.

What is the token cost pricing strategy when reusing a cached block of context?

Cached input tokens cost a fraction of the price of normal input tokens, drastically reducing costs for long sessions.



Cached tokens are billed at double cost.

Cached tokens are free.

Pricing remains the same as standard input tokens.

Explanation & Rationale

Anthropic offers prompt caching where read operations from the cache are priced significantly cheaper than standard input processing.

What is the default behavior of the Messages API if no system parameter is supplied in the request?

The model generates a response without any system prompt styling, behaving as a standard chatbot.



The request fails with an HTTP 400 error code.

The model defaults to a pre-defined system persona.

The api-version is set to legacy.

Explanation & Rationale

The system parameter is optional. If left out, Claude processes the user messages array using its default chat weights.

Which model of the Claude 3.5 family should be used for high-intelligence coding assignments, multi-stage reasoning, and data analysis tasks?

Claude 3.5 Haiku

Claude 3.5 Sonnet



Claude 3.0 Opus

Claude 2.0

Explanation & Rationale

Claude 3.5 Sonnet represents the highest intelligence model in the 3.5 lineup, optimized for complex engineering, math, and reasoning.

What is the token pricing structure of input vs. output tokens for Claude models?

Input tokens are cheaper than output tokens, making it efficient to supply rich context frames.



Input tokens cost the exact same as output tokens.

Output tokens are cheaper than input tokens.

Billing is based strictly on latency, not token count.

Explanation & Rationale

Like most LLM APIs, Anthropic bills output tokens (generation) at a higher rate per token than input tokens (prompt context).

Which parameter controls the maximum length of the dynamic system prompt?

There is no parameter limiting the system prompt; it is only bound by the overall 200k token context window of the model.



max_system_tokens

limit_system

system_tokens_limit

Explanation & Rationale

System prompts are part of the input context, limited only by the overall input context window of 200,000 tokens.

When sending dynamic user messages using the Messages API, which data type is required for the ``content`` parameter?

A raw string or an array of content blocks (such as text blocks or image blocks). 

Strictly a base64 encoded string.

Strictly a JSON string.

An array of strings.

Explanation & Rationale

The Messages API supports both raw text strings and structured arrays of content blocks (for multimodality or structured tool responses).

You are building an application that handles large file uploads. What is the maximum input context window size supported by Claude 3.5 Sonnet?

100,000 tokens

200,000 tokens



80,000 tokens

500,000 tokens

Explanation & Rationale

Claude 3.5 Sonnet features a 200,000 token context window, allowing developers to process extensive files, code repositories, or histories.

When utilizing prompt caching in the Messages API, which metadata blocks are eligible to receive cache flags?

The system parameter, tools configuration block, and dynamic items inside the messages array.



Only the model name parameter.

Only user messages containing file attachments.

The authorization header segment.

Explanation & Rationale

You can cache system prompts, tool declarations, or historical turns within the messages array by attaching cache control flags.

Which Messages API parameter allows you to establish structural context, instructions, and rules before the conversation history begins?

messages[0].content

system



prompt_prefix

model_rules

Explanation & Rationale

The 'system' parameter is used to specify system prompts. This isolates structural rules from the user messages array.

What happens if the model reaches the token count defined by the ``max_tokens`` parameter before completing its response?

The API triggers an HTTP 500 error code.

The response is truncated and the `stop_reason` is set to 'max_tokens'.



The API automatically bills for double the usage and completes the run.

The prompt is automatically re-run with lower temperature.

Explanation & Rationale

If ``max_tokens`` is hit, the model stops immediately. The resulting API payload has a `stop_reason`` of ``max_tokens`` indicating truncation.

When implementing streaming via the Messages API, which stream event containing metadata signals the creation of the message object?

message_start



message_delta

content_block_start

stream_opened

Explanation & Rationale

The 'message_start' event is the first event in the stream and contains the root message object, including the model name and role details.

When using the Messages API, what parameter specifies the output token budget limit for Claude's response generation?

`max_tokens`



`limit_tokens`

`output_token_limit`

`response_budget`

Explanation & Rationale

The 'max_tokens' parameter dictates the maximum number of output tokens Claude is allowed to generate before halting.

While parsing a streaming response from the Messages API, which event type contains the actual text delta additions?

content_block_start

content_block_delta



message_stop

message_delta

Explanation & Rationale

The 'content_block_delta' event contains the text fragment update (e.g., inside delta.text) as the model outputs tokens.

You want Claude to return a tool call arguments array, but want to strictly block it from outputting parallel tool calls in the same turn. What setting resolves this?

Parallel tool calls cannot be disabled programmatically; developers must check and reject extra calls inside the client validation logic.



Set ``parallel_tools: false`` in headers.

Set ``tool_choice: { type: "single" }``.

Lower the temperature to 0.0.

Explanation & Rationale

The API does not have a flag to disable parallel tool calls. If parallel outputs are not supported by your backend, you must handle and validate them client-side.

What schema format is officially supported by Anthropic to declare tool parameters in the Messages API payload?

XML Schema definition (XSD)

JSON Schema draft-07



GraphQL Type Schema

TypeScript Interface declarations

Explanation & Rationale

Anthropic utilizes standard JSON Schema formats to define the structure, properties, and constraints of parameters passed to tools.

When Claude decides to invoke a tool, what is the exact structure of the ``content`` block in the assistant message returned by the Messages API?

An array containing a content block with ``type: "tool_use"``, a unique ``id``, a ``name``, and an ``input`` object containing the arguments. ✓

A raw string containing Python code.

A JSON string inside a `<tool>` XML tag block.

An HTTP request payload pointing to the external server endpoint.

Explanation & Rationale

Claude communicates tool invocation requests via a structured object of type `'tool_use'` containing the tool's schema ID, name, and input arguments.

When configuring a tool schema in the Messages API, how do you define properties that the model **MUST** supply during tool calls?

Specify them in the prompt instruction text.

Add their property keys to the 'required' array in the parameters JSON schema. ✓

Set the property type to 'mandatory'.

Prefix the property names with an underscore.

Explanation & Rationale

JSON Schema standard dictates that required parameters must be listed inside the 'required' array of the properties object.

When a tool execution fails inside your application, how should you communicate the error message back to Claude?

Send a message with role 'user', containing a 'tool_result' block with 'is_error: true' and the error details inside the 'content'.



Raise an HTTP 500 error code to Anthropic servers.

Send the error message inside the system prompt.

Shut down the chat thread.

Explanation & Rationale

Anthropic's tool result schema supports an 'is_error' boolean flag. Setting this flag to true tells Claude the execution failed, allowing it to adapt its logic.

You want to design a tool schema that accepts an optional enum parameter. How should you specify this parameter in the properties block of your tool definition?

`{ "type": "string", "enum": ["high", "medium", "low"] }` without adding it to the required list. 

`{ "type": "string", "options": ["high", "medium", "low"], "required": false }`

`{ "type": "string", "default": "medium" }` inside the system prompt.

Tool schemas do not support optional enum keys.

Explanation & Rationale

Standard JSON Schema represents optional parameters by leaving their keys out of the tool's 'required' list, and restricting options using the 'enum' property.

When Claude returns a tool execution request, does it perform the execution logic automatically?

No, Claude only output arguments and requests tool invocation. The developer's client application must execute the tool and send back the results.



Yes, Claude runs Python scripts internally to resolve tool requests.

Yes, Anthropic hosts serverless execution environments for tools.

No, tool execution is resolved by the browser runtime.

Explanation & Rationale

Claude does not execute tools. It determines *when* a tool should be run and *what* parameters to use, and delegates execution to the client application.

You want to force Claude to call a specific tool in your tool array, regardless of what the user says. Which Messages API field controls this?

`tool_choice: { type: "tool", name: "my_specific_tool" }`



`required_tool: "my_specific_tool"`

`force_call: true`

`tool_choice: { type: "any" }`

Explanation & Rationale

Setting `tool_choice` to type tool` with the exact tool name forces Claude to output arguments for that tool instead of a text turn.`

You are defining a JSON Schema for a tool called `fetch_records`. It has a parameter `record_ids` that must contain exactly an array of integers. What is the correct schema structure?

```
{ "type": "array", "items": { "type": "integer" } }
```



```
{ "type": "integer[]" }
```

```
{ "type": "array[int]" }
```

```
{ "type": "list", "items": { "type": "int" } }
```

Explanation & Rationale

JSON Schema represents arrays using 'type': 'array', and items inside the array are specified via the 'items' object containing the element type.

After Claude outputs a tool request, how must your application submit the tool execution results back to the Messages API?

Add a message to the messages array with the role 'user', containing a content block of type 'tool_result' that matches the 'tool_use_id'.



Call a separate HTTP POST endpoint `/v1/tool_results`.

Inject the result into the system prompt parameters.

Modify the previous assistant message payload directly.

Explanation & Rationale

Tool results are submitted by appending a 'user' message to the thread containing a content block with type 'tool_result' and the matching 'tool_use_id'.

If you want to prevent Claude from returning any text explanation alongside a tool call request, what parameter settings can assist?

Set `tool_choice` to type `tool` specifying the target tool name.



Set model temperature to 1.0.

Add a system prompt saying 'Only run tools, do not speak.'

There is no mechanism to prevent conversational preambles.



Explanation & Rationale

Forcing tool usage via `tool_choice: { type: "tool", name: "..."}` stops Claude from generating conversational preamble text.

You are defining a tool schema for a tool named `fetch_logs`. It requires a parameter `log_levels` which should be an array of strings (e.g., `['info', 'error']`). How do you specify this in JSON Schema?

```
{ "type": "array", "items": { "type": "string" } }
```



```
{ "type": "string[]" }
```

```
{ "type": "list", "contents": "string" }
```

```
{ "type": "array[string]" }
```

Explanation & Rationale

JSON Schema represents lists as `'type': 'array'`, and uses the `'items'` key to define the constraints on elements within that array.

To allow Claude to use multiple custom tools during a single response turn, what schema configuration adjustments are needed?

No changes are needed; Claude 3.5 Sonnet supports multiple tool call responses in a single turn by default if they are declared in the tools array.



Set ``allow_parallel_tools`` to true in the API header.

Each tool must be wrapped in a separate HTTP request payload.

Configure ``tool_choice`` to 'parallel_forced'.

Explanation & Rationale

Claude models natively support parallel tool use, allowing them to output multiple 'tool_use' blocks in a single content array turn.

If you declare multiple parameters in your tool's properties JSON, how do you make sure the model supplies all of them?

List all property keys inside the 'required' array of the JSON Schema.



Include 'MUST INCLUDE ALL' in the system prompt.

Tool parameters are always optional.

Enable parallel tool options.

Explanation & Rationale

In JSON Schema, the properties defined are only mandatory if their keys are added to the 'required' list of the root parameters schema.

How should an application handle a scenario where Claude returns a tool call request for a tool that is not in the declared tool array?

Treat it as an execution bug, return a failure message block back to Claude, and log the incident.



Attempt to execute a random script matching the name.

Instantly shut down the server.

Change the model parameter to Opus.

Explanation & Rationale

If Claude hallucinations target non-existent tools, developers should return an error block in the next turn to allow self-correction.

When defining tool descriptions in your JSON Schema, what is the best practice to help Claude select the correct tool?

Provide highly descriptive, clear, and unambiguous functional statements detailing what the tool does and when to call it.



Keep descriptions as short as possible to save tokens.

Write descriptions in uppercase.

Describe the tool code logic inside the system prompt instead.

Explanation & Rationale

Claude relies on tool descriptions and parameter descriptions to understand the tool's purpose and choose the correct parameters.

How can you instruct Claude to output a valid XML document as its completion response?

Set temperature to 0.0.

Prefill the assistant response block with '<root>' to start the completion format. ✓

Wrap the system prompt in brackets.

Use base64 encoding.

Explanation & Rationale

Prefilling the assistant response starting token with `<root>` forces the model to complete the tag structure, forcing XML formatting.

To ensure that Claude outputs raw JSON without conversational preambles (like 'Here is the JSON you requested:'), what API prefilling technique is recommended?

Add 'Output JSON' to the system prompt.

Supply a message with role 'assistant' containing the opening brace '{' to prefill the response. ✓

Use temperature 0.0.

Set stop_sequences to [' '].

Explanation & Rationale

Prefilling the assistant's response with '{' forces Claude to continue the completion directly from the JSON structure, skipping preambles.

What is the recommended design pattern when using variables inside prompt templates in your software application?

Isolate variables using custom XML tags and refer to those tags specifically in your instructions.



Always write variable values in base64 format.

Combine all variables into a single long string without tags.

Place variables inside the tools array.

Explanation & Rationale

Encapsulating variables in XML tags (e.g. `<data>{{DATA}}</data>`) ensures the model can identify boundaries and comply with instructions.

You want to inject dynamic reference files into the prompt. Which XML structure represents the cleanest layout for Claude?

```
<documents><document index="1"><source>{{SOURCE}}</source><content>{{CONTENT}}</content></document></documents>
```



```
--- FILE 1 --- {{CONTENT}} --- END FILE ---
```

```
Document content: {{CONTENT}}
```

```
[[document: {{CONTENT}}]]
```

Explanation & Rationale

Wrapping multiple documents in nested XML tags allows Claude to reference documents specifically by index or metadata (e.g. 'Refer to doc 1').

You want Claude to output answers in a custom XML format. How should you prefill the assistant response to ensure it adheres to this format?

Start the assistant message with '<response>' to force the model to output content matching your XML tags.



Specify '<xml>' in the model temperature field.

Add a stop sequence for '</response>'.

Inject brackets directly into the system prompt prefix array.

Explanation & Rationale

Prefilling response arrays starting with '<response>' forces Claude to continue the XML block structure without conversational headers.

How does Claude handle prompt instruction overrides (jailbreaks) that are wrapped inside XML tags compared to plain text strings?

XML tags create clear semantic boundaries, making it easier for Claude to recognize dynamic data inputs and ignore injected overrides when instructed by the system prompt.



XML tags automatically trigger encryption protocols.

Claude rejects any message containing brackets (< or >).

Claude treats XML tags as system messages with highest priority.

Explanation & Rationale

Claude is pre-trained to parse XML tags as structured document boundaries, separating instructions from user-supplied data variables.

Why does Anthropic recommend using matching closing tags (e.g., `</docs>`) inside system prompts when referencing structured data?

It signals to the model exactly where the input context ends, preventing instructions from bleeding into subsequent segments. ✓

It reduces prompt execution latency.

It activates prompt caching protocols.

It is required by the HTTP compiler to parse the JSON body.

Explanation & Rationale

Closing tags provide explicit structures, allowing the model's self-attention layers to distinguish instructions from variables clearly.

You want to dynamically test how temperature impacts the output variety. How should you design your prompt testing system in the Console Workbench?

Run multiple Workbench instances with different temperature settings under the same prompt scenario and compare outputs.



Variables are not supported for temperature values.

Temperature must be configured inside the prompt text.

Workbench only supports temperature 0.0.

Explanation & Rationale

The Console Workbench allows you to manually adjust parameters like temperature for each test run to compare results.

Why should you avoid hardcoding variable values directly into prompt templates in your software application?

It limits reuse. Separating prompt templates from dynamic variables using double curly braces allows modular development.



Hardcoded values are billed at a higher rate.

It triggers security verification alerts.

Hardcoded prompts cannot be cached.

Explanation & Rationale

Separating the static prompt template from dynamic data variables is standard practice for clean, modular agent development.

How should prompt templates handle multiline dynamic data, such as code segments or files, inside prompts?

Wrap the dynamic string in dedicated XML tags (e.g. `<code_block>{{CODE_BLOCK}}</code_block>`) to prevent formatting leaks.



Replace all newline characters with empty spaces.

Encode the strings into base64 format.

Inject multiline files directly without delimiters.

Explanation & Rationale

XML tags handle multiline text gracefully, isolating content from instructions and preventing formatting leaks.

You want to prefill the model response to guarantee JSON structure output. What sequence of keys represents the correct payload setup in the messages array?

Last message: { role: "assistant", content: "{ " }



System prompt: "Output raw JSON starting with {"

First message: { role: "system", content: "{ " }

Header parameter: { format: "json" }

Explanation & Rationale

Ending the messages array with an assistant message containing the opening character `{` forces the model to complete the JSON payload directly.

In the Claude Console Workbench, how are prompt variables (such as `{{CLIENT_NAME}}`) evaluated during interactive testing?

They are automatically replaced by input fields in the user interface, allowing you to run test scenarios with different values.



They are treated as plain text strings and are not parsed.

They must be configured in a separate database table.

They require a Node.js compiler to run.

Explanation & Rationale

The Console dynamically parses double curly braces `{{...}}` as prompt variables and displays input fields for testing.

You are setting up variables in the Claude Console Workbench. What syntax defines a variable placeholder in the prompt template?

`{{VARIABLE_NAME}}`



`$VARIABLE_NAME`

`[VARIABLE_NAME]`

`{VARIABLE_NAME}`

Explanation & Rationale

Double curly braces `{{...}}` are used in the Workbench console to declare prompt variables.

You are designing a chatbot. You want to instruct Claude to adopt a specific communication style. Where should these guidelines be defined?

Inside the system parameter.



Inside the model temperature parameter.

At the beginning of each user message.

In the HTTP request headers.

Explanation & Rationale

System prompts are designed to hold formatting guidelines, rules, stylistic limits, and conversational guidelines.

You need to inject dynamic user-supplied data into a prompt template, but want to prevent Claude from interpreting the data as instruction overrides. What is the cleanest isolation format?

Put the user data inside double quotes.

Enclose the variable in XML tags (e.g., `<user_input>{{USER_INPUT}}</user_input>`) and refer to those tags in the system prompt. ✓

Convert the user data into lowercase.

Remove all space characters from the input string.

Explanation & Rationale

Isolating raw data inside XML tags and explicitly referencing those tags (e.g., 'Analyze only the contents within `<user_input>` tags') prevents prompt injections.

Your application throws an error indicating 'anthropic.APIConnectionError'. What does this represent, and how should it be handled?

The client API key has expired.

A network connectivity issue occurred while reaching Anthropic servers. Catch the error and retry using backoff. ✓

The requested model name was typed incorrectly.

The request payload exceeded token limits.

Explanation & Rationale

`APIConnectionError` indicates network transit failures. Implementing retries with exponential backoff helps resolve transient drops.

What is the primary difference between a 429 rate limit error and a 529 server overloaded error in the Anthropic API?

429 means your client exceeded request or token limits; 529 means Anthropic's servers are experiencing high traffic.



429 means an invalid key; 529 means the payload was too large.

Both represent the same error condition.

429 is a client validation error; 529 is a billing failure.

Explanation & Rationale

429 indicates client quota limits were hit. 529 represents internal server overload, which is resolved by server scaling.

Your backend app gets a '504 Gateway Timeout' error when sending an extremely complex prompt to Claude. What is the first troubleshooting step?

Assume the API key has expired and request a new one.

Review client-side network timeout configurations and check if the prompt's complexity is stalling token generation.



Immediately double the size of the request batch.

Switch to using Opus for faster speeds.

Explanation & Rationale

A 504 error typically means the proxy or client closed the connection before the API responded. Adjusting timeouts or moving to a faster model (like Sonnet/

How should a developer monitor rate limits and remaining usage quotas programmatically?

Inspect rate limit headers (such as `anthropic-ratelimit-requests-remaining` and `anthropic-ratelimit-tokens-remaining`) returned in the HTTP response.



Call the `/v1/rate_limits` endpoint hourly.

Rate limits are only visible in the Console dashboard.

Estimate usage client-side by multiplying character counts.

Explanation & Rationale

The Anthropic API returns headers indicating current rate limits, remaining requests, and remaining tokens after each call.

If your Messages API response returns ``stop_reason: "stop_sequence"``, what does this indicate about the text generation process?

The model encountered an error during generation.

The model encountered one of the custom sequences defined in the ``stop_sequences`` array parameter and halted output.



The token window limit was exhausted.

The assistant turn completed normally (``end_turn``).

Explanation & Rationale

When one of your custom-defined stop strings (such as a newline or custom tag) is generated, Claude halts execution and reports 'stop_sequence'.

Your application receives an HTTP 429 error code from the Messages API. What does this signify, and how should it be managed?

Authentication failed; refresh the API key.

Rate limit exceeded; implement exponential backoff with jitter and retry.



The prompt was too long; truncate the token counts.

Internal server error; report the issue to Anthropic.

Explanation & Rationale

429 represents rate limits. Retry strategies using exponential backoff with randomized jitter distribute requests smoothly over time.

What does an HTTP status code of 403 represent when communicating with the Anthropic Messages API?

The request was throttled due to rate limits.

Authentication failed or client credentials/API keys are invalid.



The server is temporarily overloaded.

The model name specified does not exist.

Explanation & Rationale

403 Forbidden indicates authorization failures, such as invalid or expired API keys.

What client-side headers must be supplied to authenticate calls to the Anthropic Messages API?

Authorization: Bearer <key>

x-api-key: <key> and anthropic-version: 2023-06-01



api-token: <key>

x-anthropic-auth: Bearer <key>

Explanation & Rationale

Anthropic API requests require the 'x-api-key' header for credentials and the 'anthropic-version' header specifying the API schema version.

When parsing tool execution arguments generated by Claude, your code encounters a JSON syntax error. How should your application handle this gracefully?

Crash and report to the user.

Intercept the error and send a corrective message back to the assistant (e.g., 'Error: Invalid JSON format. Please output valid arguments.') to allow the model to self-correct.



Retry the request using temperature 1.0.

Use standard regex to strip all braces.

Explanation & Rationale

Providing structural error logs back to the model in the next turn is a standard self-healing agent pattern, allowing it to regenerate correct syntax.

Which header version flag is currently required to access the Anthropic Messages API?

anthropic-version: 2023-06-01



anthropic-version: v3.5

api-version: 2024-01-01

x-version: standard-latest

Explanation & Rationale

The API version header value '2023-06-01' is required to call Messages API endpoints.

If you receive a 401 status code from the Messages API, what is the most likely cause?

The API key supplied is invalid, missing, or typed incorrectly.



Your rate limits were exceeded.

The server is temporarily overloaded.

The payload contains malformed JSON.

Explanation & Rationale

An HTTP 401 Unauthorized status indicates that the request lacked valid authentication credentials.

Your application throws an HTTP 400 error indicating 'max_tokens' parameter is missing. What does this indicate?

You did not specify the max_tokens value in the request body.



The input prompt exceeded the max context limit.

The value of max_tokens was too high.

The API key was rate limited.

Explanation & Rationale

The 'max_tokens' parameter is a required field in every Messages API POST request.

What HTTP status code is returned if your Messages API payload contains an invalid model parameter name?

400 Bad Request (invalid_request_error)



404 Not Found

401 Unauthorized

500 Internal Server Error

Explanation & Rationale

Typing errors in parameters (such as invalid model names) return an HTTP 400 Bad Request representing validation failures.

Your server receives a '529 Overloaded' error from the Anthropic API. What does this indicate, and what action should your client-side integration perform?

The request size is too large; reduce the token count.

The Anthropic servers are experiencing high traffic; retry the request after a delay using exponential backoff.



The API key has been throttled permanently.

The model has timed out; switch to Opus.

Explanation & Rationale

529 signifies temporary server overload. Clients should handle this gracefully by retrying the request with exponential backoff and jitter.

Your API call fails with a 400 Bad Request error. The error message indicates 'invalid_request_error: system: System prompt must be a string or array of content blocks'. What payload formatting issue caused this?

The 'system' parameter was supplied inside the messages array instead of at the root level of the payload. ✓

The system prompt exceeded the token limit.

The messages array contained a system role. ✗

You did not specify a model parameter.

Explanation & Rationale

The 'system' parameter is a root-level property of the Messages API request body, not an entry in the 'messages' array.